

Boot sequence

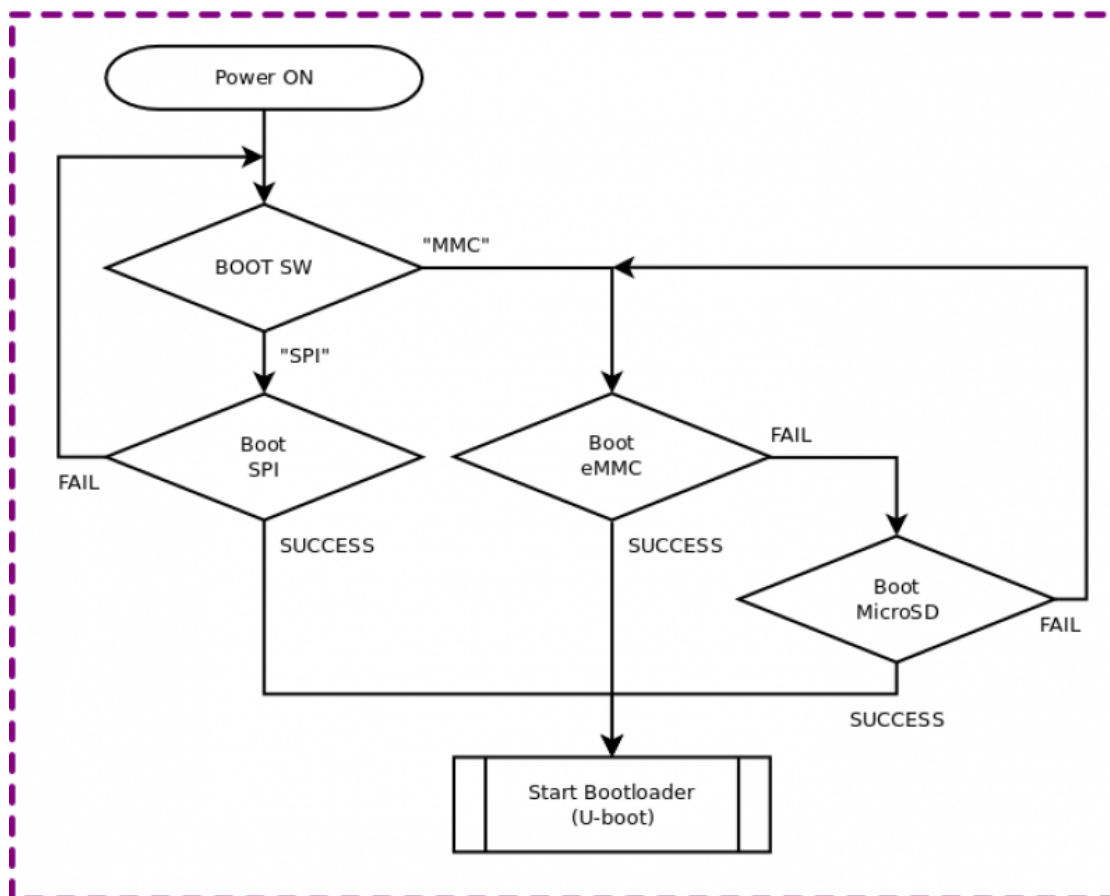
The boot selector switch on ODROID-N2 which can select the boot media between SPI flash memory and eMMC/SD card is only affected on power cycle.

Stage #1: Power cycle

This stage is aimed to fetch the bootloader software from the storage device mounted on or inserted to the board. ODROID-N2 offers three different boot storages, please visit this page to figure out the location. Especially, ODROID-N2 is the first SBC board with onboard SPI flash memory among Hardkernel's ODROID boards.

- eMMC : Removable (custom design by Hardkernel, eMMC 5.0)
- Micro SD : Removable (UHS-I)
- SPI : On-board (NOR, 8MiB)

ODROID-N2 has “Boot Select Switch” which is to select SPI flash memory as the primary boot device or not. If the switch nob is located to “SPI”, then CPU will fetch the bootloader from SPI flash memory. On the other hand, when the switch nob is placed to “MMC”, the bootloader in the eMMC or the Micro SD will be fetched by CPU. Since the eMMC has a high priority, CPU will fetch from Micro SD if eMMC does not have the bootloader.



Stage #2: Bootloader

The second stage to fetch the relatively huge software binaries from attached storage that can be accessible by the bootloader loaded at stage #1. In this stage, the system becomes more flexible to access various storage types and vendors. Even the `OS()` software can be loaded through network protocol or USB storages. Also, the bootloader also offers a command line shell so a user can run commands manually or set/store the bootloader variables.

Boot from SPI

As shown in stage #1, this stage also checks whether SPI flash memory is selected to boot from. If so, the bootloader loads the software binaries from the SPI flash memory instantly. Currently, the Petitboot (<https://mirrors.edge.kernel.org/pub/linux/kernel/people/geoff/petitboot/petitboot.html>) is ported to ODROID-N2 as a default software in the SPI flash memory. Please visit these pages if you like to read more information about Petitboot from Hardkernel.

- Petitboot

Boot from eMMC/Micro SD

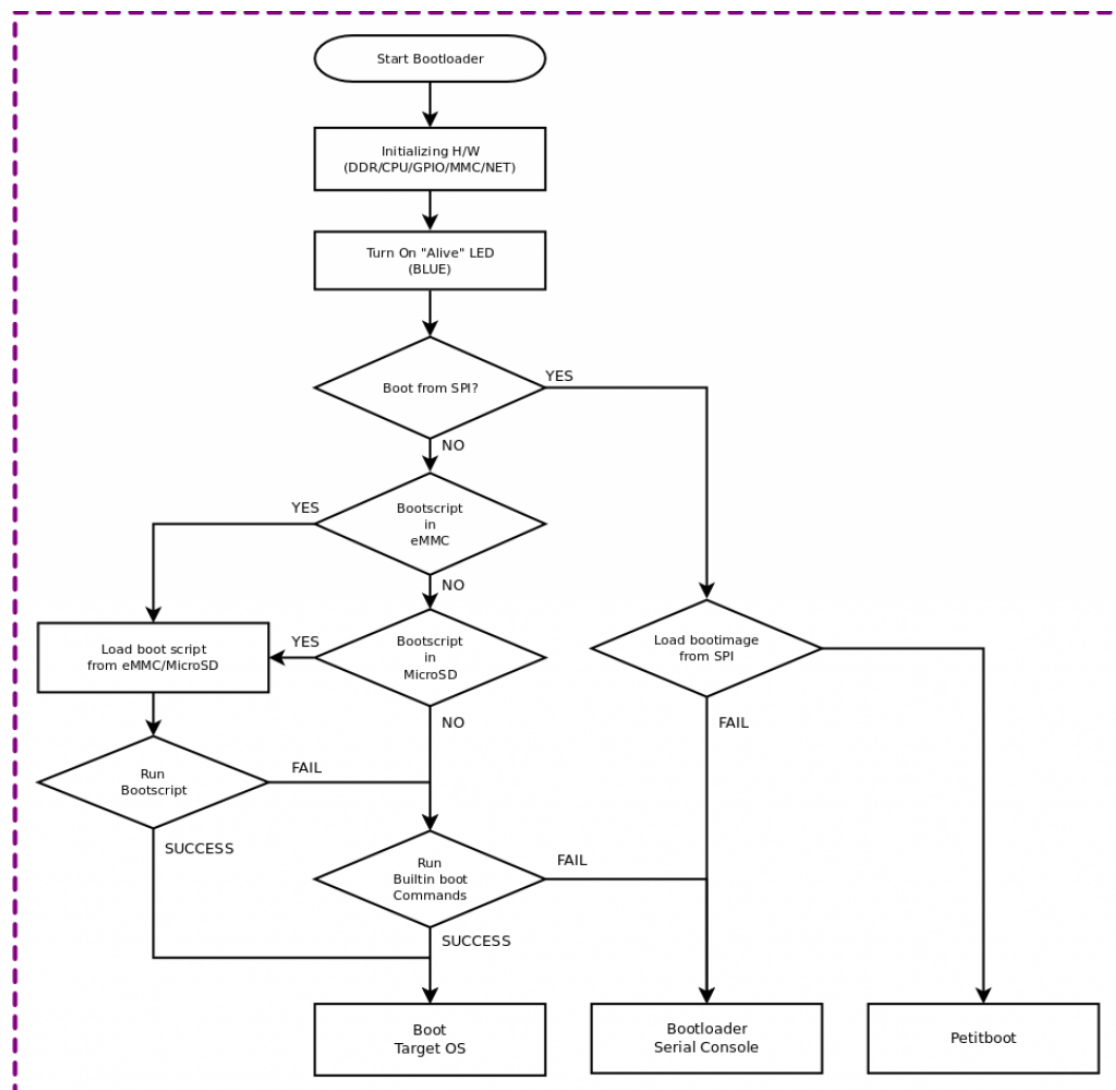
If ODROID-N2 is set to boot from eMMC or Micro SD, the bootloader looks up the first partition of each storage device from eMMC and Micro SD in order to find out the boot script. Actually, the contents in the boot script are the set of commands supported by the bootloader, U-boot, such as `setenv`, `load` and `bootm`. The current bootloader supports two types of boot scripts.

boot.ini

This is a purely text-based file, human-readable and editable by any text editor so very handy to change its contents at any platform. This script type is traditionally used by Hardkernel's ODROID SBCs. If any text file with `boot.ini` contains the board specific magic string, `ODROIDN2-BOOT-CONFIG` for ODROID-N2, at the first line, the bootloader starts to execute the script.

boot.scr

This is a legacy script format of U-boot which contains the 64 bytes binary header and test-based commands. Since this file is also editable in a text editor, one can change the commands in it. But, unlike `boot.ini`, `boot.scr` has the checksum at the header in the first 64 bytes, the header must be regenerated by the tool `mkimage` which is one of U-boot tools whenever the contents of the script. This script is not handy but can prevent to be modified and executed unexpectedly.



Debugging at U-boot

As shown in the diagram, the bootloader traverses the boot storage one after another until boot successfully. For some reason, the bootloader cannot start the next software, expensive OS () like Linux, it stops and gives a shell prompt through the serial port. With USB-UART Module Kit (<https://www.hardkernel.com/shop/usb-uart-module-kit/>), one can read logging message from U-boot or gives commands directly.

Builtin boot commands

The one may be curious what is the builtin boot command and what commands are. Basically, this is the command set to read various blobs from the boot memory card among eMMC or Micro SD that the bootloader is loaded and practically to boot Android platform that requires multiple blobs in different sizes and types more than 4 partitions which is the amount of the primary partitions in MBR (Master Boot Record). So in order to store numerous blobs for Android in the memory card, the partition table is designed and noted in the U-boot code. This partitions can be updated through the `fastboot` that is a popular tool to update Android system through a USB cable. So technically, if you have software blobs that will store in the memory card as a raw format other than a file in a file system, you could design the partition table and update them in U-boot. This page introduces the current partition table for Android.

 `odroid-n2/software/boot_sequence.txt`  Last modified: 2020/07/15 14:00 by  john1117

ODROID Wiki



Except where otherwise noted, content on this wiki is licensed under the following license:
CC Attribution-Share Alike 4.0 International